



4

24

2880 SP

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Linked List

عملي مشترك



خوارزميات

RB Informatix; 12/11/2024

السلام عليكم ورحمة الله وبركاته

مرحبا أصدقائي في محاضرة جديدة سنتعلم فيها أشياء جديدة سوية :

عناوين محاضرة اليوم :

بنى المعطيات (Data Structures) - ال Array - ال ArrayList - ال Linked List و العمليات عليها و أنواعها.

بنى المعطيات Data Structures

ما هي ال Data structures ؟

هي طريقة مجردة لتمثيل البيانات و إدارتها (سواء كانت هذه البيانات عبارة عن int - string - float - object ...)
هنا لا نهتم بنوع البيانات بل نهتم بطريقة تمثيلها مع بعضها البعض و استخدامها للذاكرة.

أهمية ال Data Structures :

1. كفاءة العمل :

يوجد عدة أشكال للـ Data structures ، كل منها له نقاط قوة و نقاط ضعف ، لذا يجب اختيار البنية الأمثل لكل مسألة تواجهنا و هذا سيؤدي إلى عمل خوارزمية الحل بالشكل الأمثل .

2. إدارة الذاكرة :

حيث أن تمثيل البيانات ببنى المعطيات المناسبة لها سيؤدي إلى تقليل هدر المساحات في الذاكرة و تنظيم البيانات فيها.

أنواع بنى المعطيات:

يوجد لدينا نوعان:

1. بنى المعطيات الخطي (Linear Data Structures) :

تكون العناصر مرتبة بشكل تسلسلي (كل عنصر يلي الآخر) ، حيث يتم الوصول من أحدها إلى الآخر بشكل خطي (أو تسلسلي)

أمثلة : Array - List - Array - Linked List - Stack - Queue - ...

2. بنى المعطيات الغير خطية (Non-Linear Data structures) :

تكون العناصر هنا مرتبة بشكل هرمي، حيث يتم الوصول من عنصر واحد إلى أكثر من عنصر .
أمثلة : Graph - Tree -

العمليات على بنى المعطيات:

إضافة (Insert) - حذف (Delete) - قلب (Reverse) - بحث (Search) - ...

المصفوفات Array

جميعنا تعاملنا مع هذا النوع من الـ data structure ، اليس كذلك ؟

تعريف: و هي عبارة عن data structure خطية، تخزن العناصر بحيث تكون متتالية في الذاكرة.

- أحد أهم مميزاتا أن تعقيد الوصول فيها إلى أي عنصر هو $O(1)$.

مثال: $arr[15] \leftarrow$ هنا نصل إلى العنصر ذو الـ index 15 بـ $O(1)$ فقط.

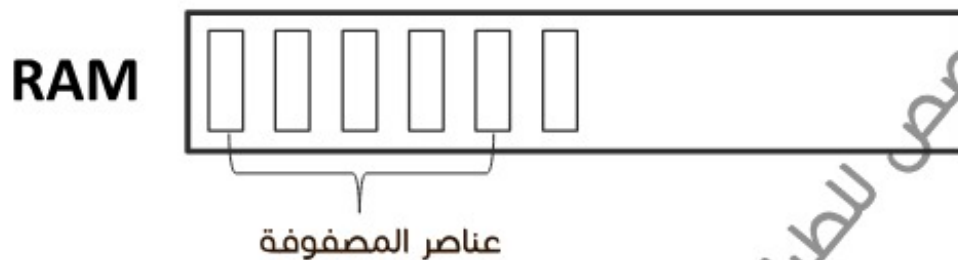
- هي ميزة جميلة جداً عندما تحتاج مسألتنا إلى قراءة الـ data أو الوصول إليها بشكل كبير.

- أحد مساوئها هي الحجم الثابت fixed size ، فلا نستطيع التغيير على حجم المصفوفة بالزيادة أو النقصان بكامل الكود.

التمثيل في ال RAM :

في ال ++c :

يتم تمثيل ال Array (سواء كانت one-dimensional array أو Multi-Dimensional Array) بشكل single-dimensional array في ال heap .

مثال: `int arr[6]`

كيف يتم حجز مساحة لهذه المصفوفة ؟

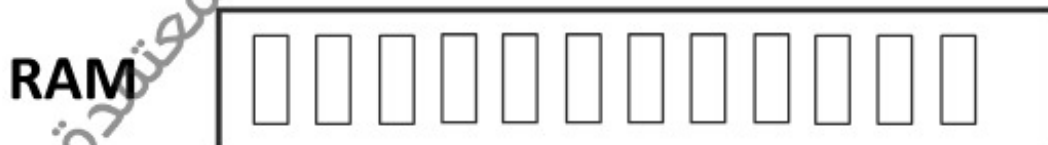
يتم ضرب حجم المصفوفة (و هو 6 في مثالنا) بحجم العنصر الواحد فيها (في مثالنا المصفوفة مؤلفة من عناصر integer أي أن حجم العنصر هو 4 byte).

$$24 \text{ byte} \rightarrow 6 \times 4 \text{ byte}$$

و ستكون هذه ال bytes متتالية في الذاكرة كما ذكرنا سابقا.

إذا أردنا الوصول إلى العنصر `arr[i]` ، كيف يتم ذلك ؟

عنوان أول عنصر في المصفوفة + (i * حجم العنصر الواحد في المصفوفة)
أي : $\&\text{arr}[0] + i * 4$

مثال: `int arr[3][4]`

■ نلاحظ أنه تم تمثيل هذه المصفوفة الثنائية كمصفوفة أحادية في الذاكرة، حيث تم حجز المساحة كالتالي:

$$(3 * 4 * 4 \text{ byte})$$

" حيث أن ($3 * 4$) يمثل عدد عناصر المصفوفة و 4 byte هو حجم العنصر الواحد من نوع int ."

ملاحظة أريسيزية:

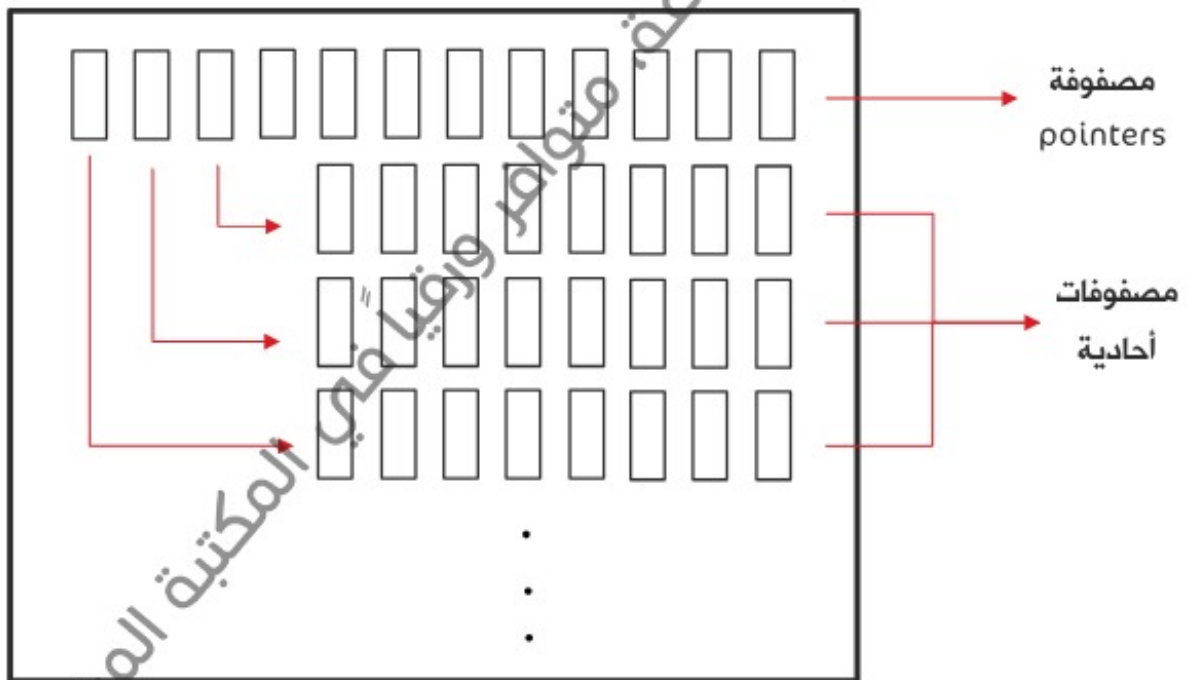
الوصول إلى العنصر $arr[i][j]$ في المصفوفة السابقة يتم كالتالي:

في مثالنا $type$ ————— $\&arr[0][0] + (i * \text{عدد الأعمدة} + j) * \text{sizeof}(type)$ ————— موقع العنصر $arr[0][0]$
 العناصر هو i عمود العنصر
 سطر العنصر المطلوب
 عدد الأعمدة في مثالنا هو 3

في الـ java :

الـ Multi-Dimensional Arrays هي عبارة عن مصفوفة من $pointers$ حيث كل $pointer$ يشير إلى مصفوفة أحادية كالتالي:

RAM



تذكرة :

الـ $pointer$ هو متغير يحمل عنوان متغير آخر في الذاكرة.



```
int main(){
    int x=3;
    cout<<x; // --> 3
    cout<<&x; // --> 0x7ffd22d884
    return 0;
}
```

ArrayList

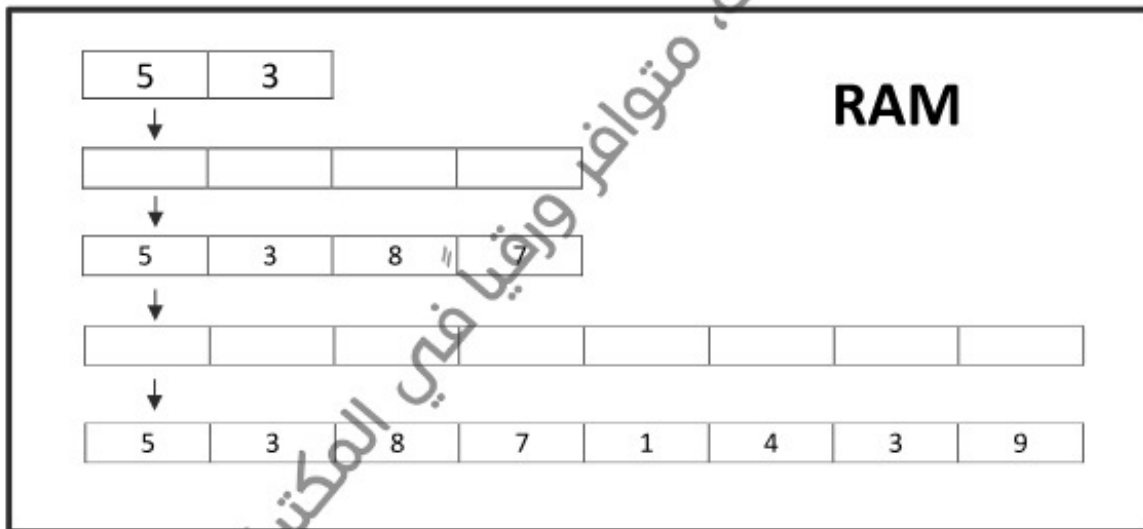
و هي عبارة عن array لكن لها dynamic size ، كيف ذلك و ما آلية عملها؟

- في البداية يكون لل array حجم أولي افتراضي، ثم عند إدخال عناصر و بفرض أن عدد العناصر أكبر من حجم ال array ، بالتالي :

- (1) سيتم إنشاء array أكبر بنسبة معينة .
- (2) نسخ عناصر ال array إلى ال array الجديدة .
- (3) إضافة العناصر الجديدة لـ array الجديدة.

- هذه العملية تكرر دائما عندما تمتلئ ال ArrayList .

• مثال :



- **ملاحظة :** هنا اعتبرنا أن نسبة تكبير ال ArrayList هي 2 (أي الضعف).

- نلاحظ أن ال ArrayList لديها dynamic size لكن عند الاستخدام الكبير فسيتم حجز مساحة كبيرة قد لا يتم استهلاكها بشكل كامل (هدر للذاكرة) .

• مثال :

- إذا كان لدينا ArrayList من 1000 عنصر و نريد أن نضيف لها عنصر واحد فقط، فسيتم إنشاء ArrayList أكبر بنسبة 1.5 أي أنها ستحتوي 1500 عنصر، بالتالي سنحجز 1500 عنصر في الذاكرة من أجل إدخال عنصر واحد فقط.

ملاحظة أريسية:

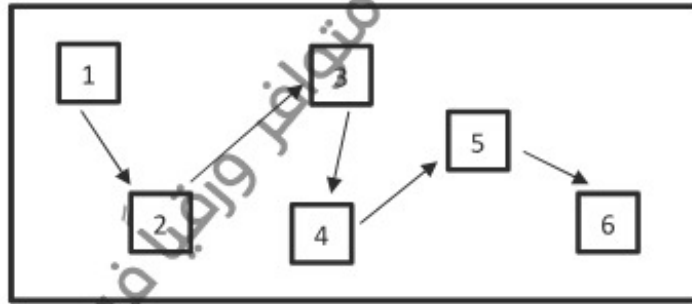
لحل مشكلة هدر المساحة يمكن استخدام التابع trimToSize() الذي يقوم بتقليص حجم الـ ArrayList حتى يصبح يساوي عدد العناصر التي تحويها

- الوصول بالـ array أسرع من الوصول بالـ ArrayList، لكن هذا الفرق بسيط جداً ويمكن إهماله، فيكون تعقيد الوصول في كلا الـ Array و الـ ArrayList هو $O(1)$.

Linked List

- هي data structure التي تسمح لنا بتخزين عناصر بأماكن عشوائية غير مرتبة في الذاكرة (أي أن العناصر ليست متتالية). كل عنصر في الـ linked list يحوي على pointer يشير إلى العنصر الذي يليه في الـ linked list . كالتالي :

RAM



كيف نقوم بذلك؟

ستكون العناصر هي عبارة عن Nodes (عقد) ، حيث أن كل عقدة تحوي على :

1. data : وهي البيانات التي نريد تخزينها أي كان نوعها.
2. next : هو pointer يشير إلى الـ node التالية.

ملاحظة : في محاضرتنا سنكتب بلغة الـ java ، لكن يمكن تطبيق الـ Linked List بأي لغة.

سيكون الكلاس كالتالي :

```

1) public class Node {
2) int data;
3) Node next;
4) public Node (int data){
5) this.data = data ;
6) }
7) }

```

شرح أريسيزي :

ال object reference هو عبارة عن متغير يعمل ك pointer في ال java حيث يشير إلى موقع ال object في الذاكرة .

مثال للتوضيح :

Object o;

ال o هنا هو pointer لا يشير إلى أي عنصر.

o=new Object();

هنا خلقنا ال object فأصبح ال o يشير إلى هذا ال object في الذاكرة

كما نعرف يا أصدقائي لغة ال java هي لغة oop ، أي أننا نتعامل مع كل شيء على أنه object و نقوم بالتعديل عليه من خلال methods .

← ستكون ال Linked List هي عبارة عن object فيه methods تسمح لنا بالتعديل على محتوياتها.

ملاحظة أريسيزية :

يمكننا استخدام ال Linked List الموجودة في اللغة by default . لكن في محاضرة اليوم سنتعلم سوية إنشاء Linked List بانفسنا و كتابة التوابع التي داخلها :

محتويات class ال Linked List :

1. **Head** : و هو عبارة عن pointer يشير إلى أول Node في ال Linked List ، و هذا العنصر مهم و حساس جداً حيث أن من خلاله نستدل على ال Linked List و نصل إلى بقية العناصر.

2. **Node ال Class** : سيكون class ال Node عبارة عن inner class داخل ال Linked List ،

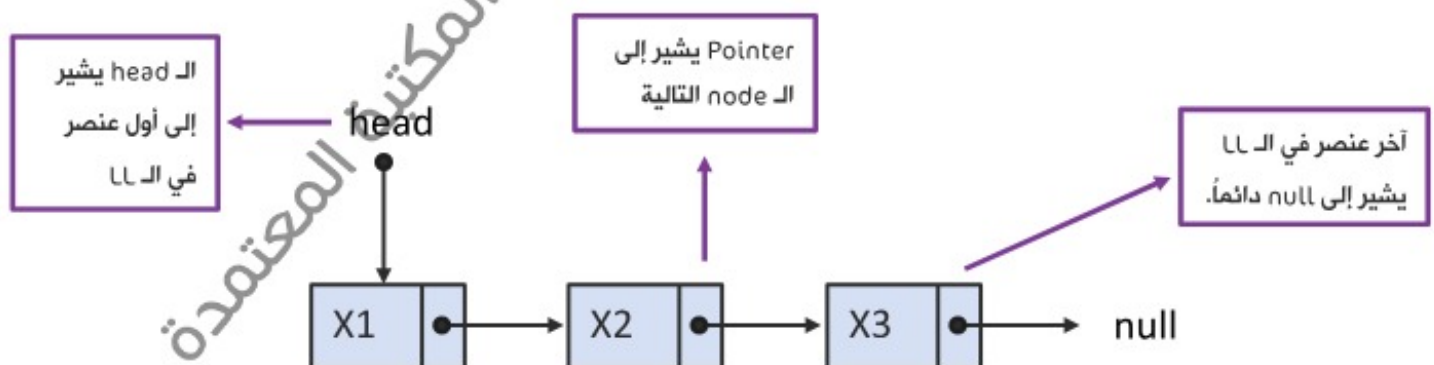
لماذا؟؟؟

"لأنه لا حاجة لاستخدام class ال Node خارج class ال Linked List ، فبجعله inner class فنحن قد قمنا بعملية تغليف (encapsulation) حيث أن المستخدم سيقوم باستخدام ال Linked List و التعديل عليها من خلال ال methods الخاصة بها دون الحاجة إلى رؤية محتوياتها".



```
1) public class LinkedList {
2)     Node head;
3)     public class Node {
4)         int data;
5)         Node next;
6)     public Node (int data){
7)         this.data = data ;
8)     }
}
```

مخطط توضيحي عن ال Linked List :



ملاحظة :

بما أنه لم أنشئ constructor لـ Linked List ← ال head سيأخذ قيمة null بشكل افتراضي عند إنشاء ال Linked List و سيتم تعديله عند إضافة عنصر باستخدام ال method التي تقوم بذلك .

الآن يا أصدقائي سنقوم بكتابة عدة توابع داخل class الـ Linked List الخاص بنا، لنفهمها سوياً ☺ :

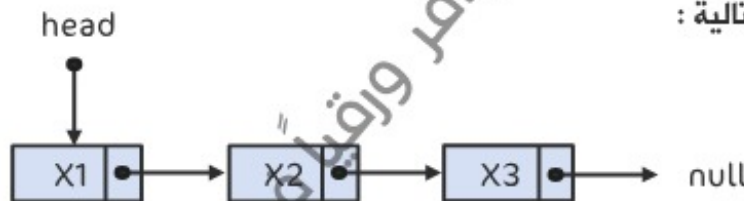
1- تابع لإضافة عنصر إلى بداية الـ Linked List :

```
public void insertFirst(int data) {
1)   Node newNode = new Node(data);
2)   newNode.next = head;
3)   head = newNode;
}
```

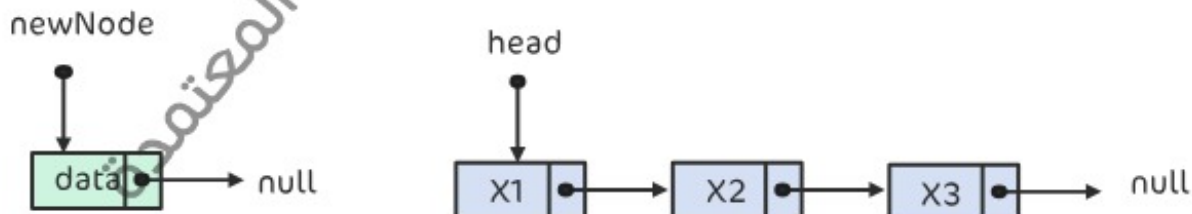
نلاحظ أن الوصول في الـ Linked List يتم باتجاه واحد دائماً (من أول عنصر إلى آخر عنصر) فكل عنصر يشير إلى الذي يليه ولا يستطيع الوصول إلى الذي قبله. بالتالي عند الوصول إلى أول عنصر في الـ Linked List يمكننا الوصول إلى بقية العناصر، وهنا تكمن أهمية الـ head .

شرح الكود :

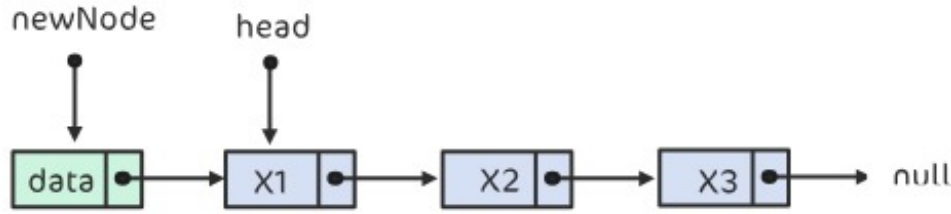
• لدينا الـ Linked List التالية :



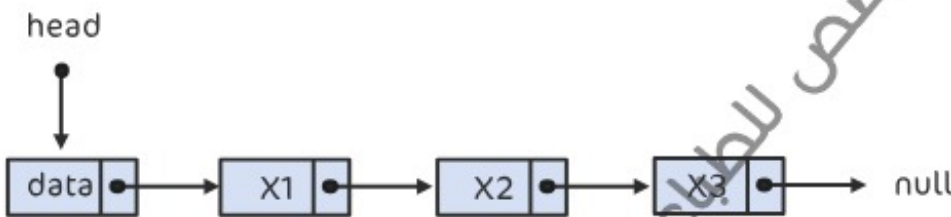
1. ننشئ الـ node الجديدة التي نريد إضافتها والتي تحوي الـ data التي تم تمريرها كـ parameter إلى التابع.



2. نجعل الـ pointer الخاص بالـ node الجديدة يشير إلى الـ node الأولى في الـ Linked List.

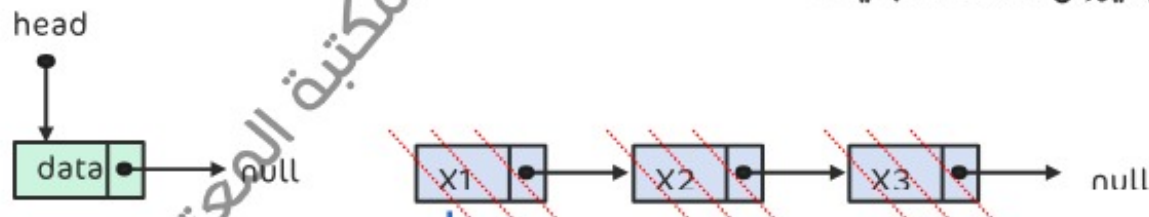


3. نجعل الـ head يشير إلى الـ node الجديدة.



ملاحظة هامة :

في الـ java يوجد garbage collector ، يقوم بإتلاف (حذف) جميع الـ objects التي لا يشير لها أي pointer ← إذا كان الـ object لا يشير إليه pointer ← سيتم حذفه .
فرضاً : لو أننا بدلنا بين الخطوة 1 و 2 في التابع السابق لكان حدث التالي :
أنشأنا الـ node الجديدة التي نريد إضافتها.
جعلنا الـ head يشير إلى الـ node الجديدة.



سيقوم الـ garbage collector بحذف هذا العنصر لأنه لا يوجد pointer يشير إليه.

سيتم حذف جميع العناصر التالية، فسنخسر الـ Linked List كاملة

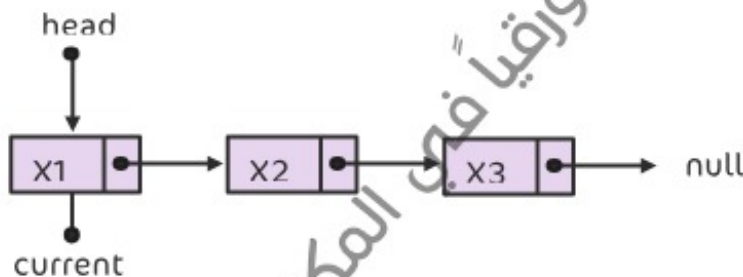
← دائماً نجعل الـ pointer يشير إلى الـ node الأولى ثم ننقل الـ head حتى لا نخسر الـ Linked List :

- نلاحظ أن تعقيد إضافة عنصر إلى بداية الـ Linked List هو $O(1)$ ، لأننا نقوم بعدد عمليات ثابتة في كل مرة (3 عمليات).
- أما لو كنا نريد إضافة العنصر في نهاية الـ Linked List فيجب علينا الوصول إلى آخر عنصر ثم إضافة الـ node الجديدة، وهذا يتطلب المرور على جميع عناصر الـ Linked List ← تعقيده $O(n)$.

2- تابع لطباعة عناصر الـ Linked List :

```
public void print(){
    1) Node current = head;
    2) while (current != null) {
    3) System.out.println(current.data);
    4) current = current.next;
    }
}
```

1 | نعرف pointer يشير إلى أول عقدة في الـ Linked List اسمه current (يمكن تسميته كما نشاء طبعا)



2 | ننشئ حلقة شرط توقفها هو $(current \neq null)$ أي أنها لن تكسر حتى يصبح الـ current يشير إلى null والذي تشير له آخر node في الـ Linked List .
و بفرض كان $head = null$ ← لن ندخل إلى الحلقة ⑥
← داخل الحلقة :

- (a) (السطر 3) : عند كل قيمة للـ current نطبع الـ data الخاصة بالـ node التي يشير إليها .
(b) (السطر 4) : نجعل الـ current ينتقل إلى العقدة التالية حتى نصل إلى نهاية الحلقة
فيصبح الـ $current = null$ و تكسر الحلقة.

- **بشكل عام :** إذا أردنا التنقل بين عناصر الـ Linked List فنقوم بإنشاء pointer يشير إلى أول عقدة فيها، و حلقة شرط توقفها أن يكون الـ pointer يساوي null.

3- تابع للبحث عن عنصر في الـ Linked List :

- سنقوم بـ Linear search في الـ Linked list :

```
public boolean search(int data){
1) Node current = head;
2) while(current != null){
3)   if(current.data == data){
       return true;
   }
4)   current = current.next;
   }
5) return false;
}
```



(1) ننشئ pointer باسم current يشير إلى الـ node الأولى.

(2) ننشئ حلقة شرط توقفها هو `current != null` (أي أنها ستمر على جميع عناصر الـ Linked List ...

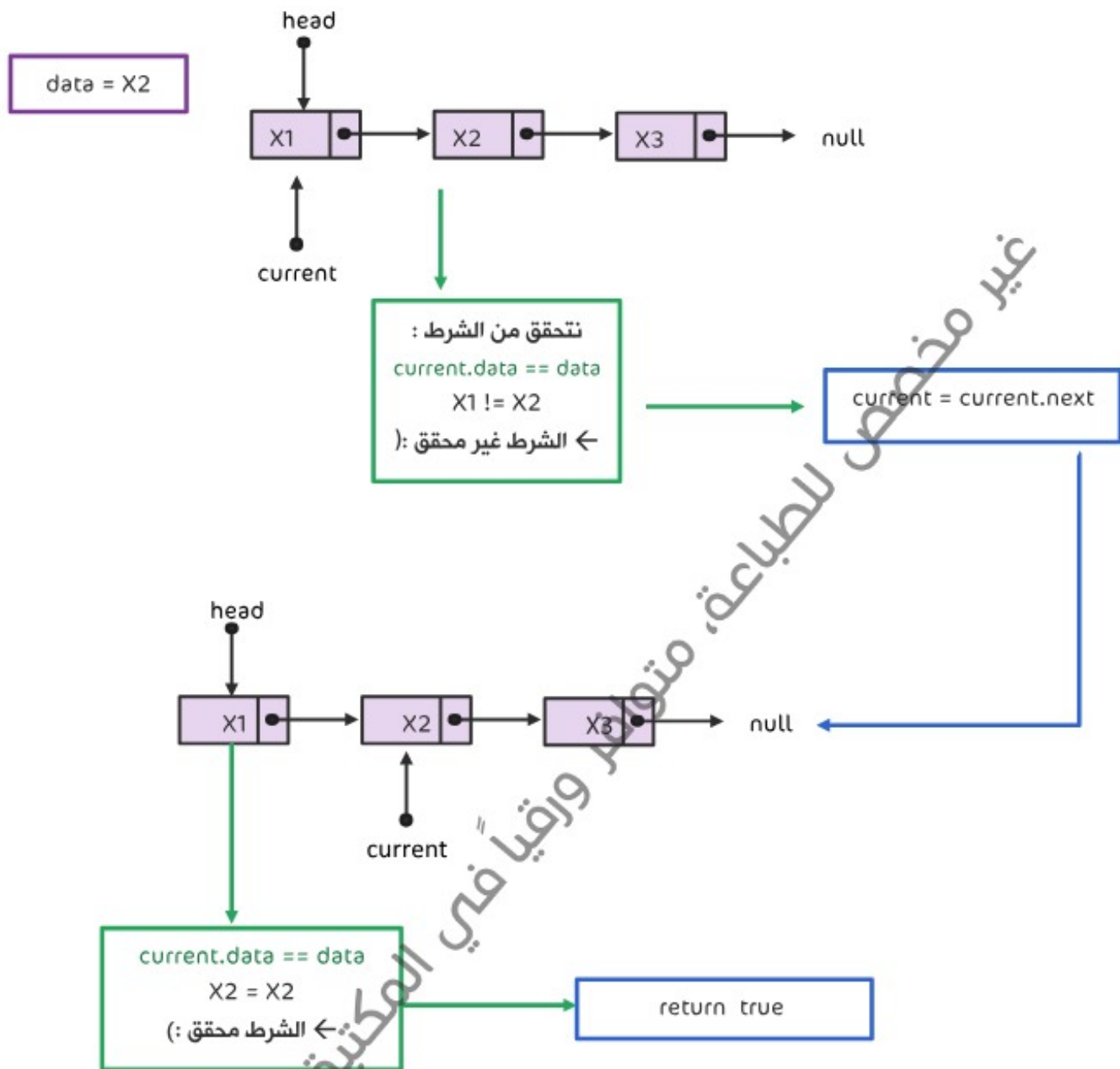
كما قمنا في التمرين الماضي (٢٤))

← داخل الحلقة :

(a) (السطر 3): نقارن الـ data الخاصة بالـ node التي نشير إليها بالـ data التي نريد البحث عنها (أي بالـ data الممررة إلى التابع كـ parameter)،

إذا كان الشرط محقق (أي أننا وجدنا العنصر المطلوب في الـ Linked List) ← يرجع التابع true .

(b) (السطر 4): نجعل الـ pointer (current) يشير إلى الـ node التالية (سينفذ هذا السطر عند عدم تحقق الشرط).



و بفرض كانت data غير موجودة في Linked List :
 ← سيمر ال current على جميع عناصر ال Linked List و الشرط لن يتحقق عند أي منها، حتى يصبح ال current يساوي null و نخرج من الحلقة ويقوم التابع بإعادة false.

4-تابع لحذف عنصر من الـ Linked List :

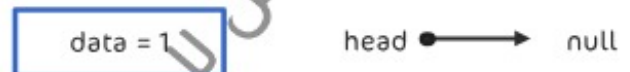
```
public void deleteOneNode(int data)
```

```
1) if (head == null) {
2)   return ;
3) }
4) if (head.data == data) {
5)   head = head.next ;
6)   return;
7) }
8) Node prev = head;
9) Node current = head.next;
10) while (current != null) {
11)   if (current.data == data) {
12)     prev.next = current.next ;
13)     return;
14)   }
15)   current = current.next ;
16)   prev = prev.next ;
17) }
18) }
```

في هذا التابع سنناقش عدة حالات سوية (٤٤) :

1. حالة إذا كان الـ head = null :

أي أن الـ Linked List فارغة فلا يوجد عناصر لحذفها
← ينتهي التابع مباشرة)



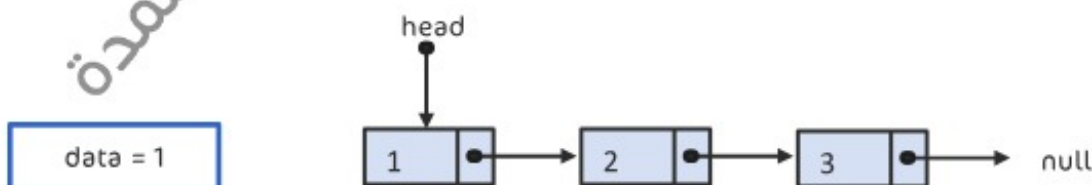
2. حالة إذا كان العنصر المراد حذفه هو أول عنصر

في الـ Linked List :

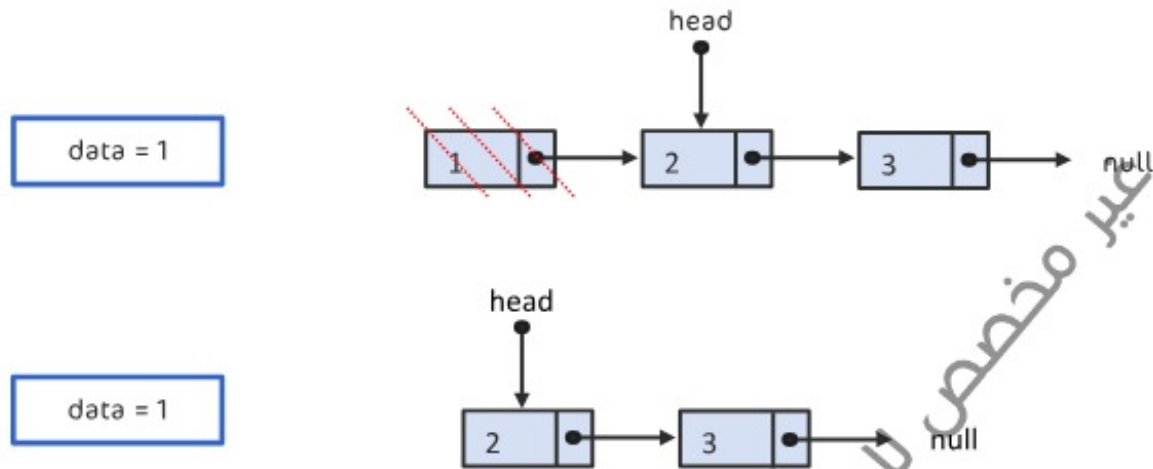
نناقش هذه الحالة بمفردها لأننا سنحتاج هنا إلى

التعامل مع الـ head مع الحرص على الـ Linked List :

- **السطر 4 :** الشرط الذي يحقق هذه الحالة (head.data == data) أن الـ data الذي يشير إليها الـ head تساوي الـ data المطلوبة.



- السطر 5 : نجعل الـ head يشير إلى الـ node التي تلي الـ node الأولى
← ستصبح الـ node الأولى بلا pointer يشير إليها ← الـ garbage collector سيقوم بحذفها.



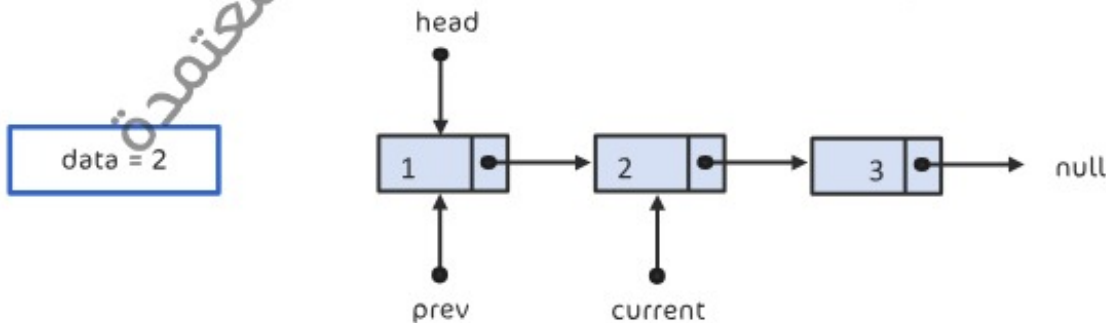
- السطر 6 : نهي التابع :

3. حالة اذا كان العنصر المراد حذفه هو أي node عند الـ node الأولى :

- السطر 8 : أنشأنا الـ pointer الذي يدعى prev و جعلناه يشير إلى الـ head.
وظيفة هذا الـ pointer هي الاحتفاظ بمعلومات الـ node السابقة، فكلما قلنا في الـ Linked List الانتقال يتم باتجاه واحد، حيث أن كل node تشير إلى التي بعدها ولا تستطيع الوصول إلى التي قبلها ← استعنا بهذا الـ pointer :
السطر 9 : أنشأنا الـ pointer الذي يدعى current و جعلناه يشير إلى الـ head.next (الـ node التي تلي الـ head)

لماذا لم نجعله يشير إلى الـ head ؟؟؟

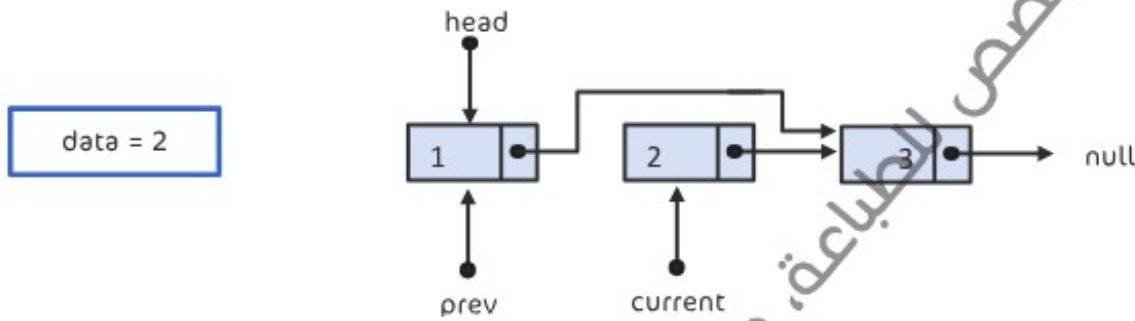
لأننا عالجت حالة الـ head سابقا ... :



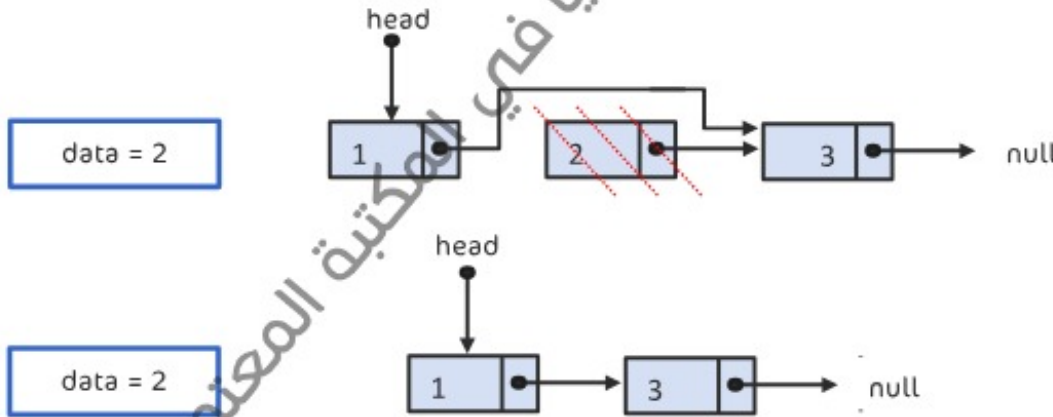
- السطر 10 : حلقة للمرور على جميع عناصر الـ Linked List داخل الحلقة (لن تنكسر حتى يصبح الـ current يشير إلى null والذي تشير له آخر node):

a. السطر 11 : الشرط `current.data == data` ، أي أنه إذا كانت الـ data التي يشير إليها الـ current تساوي الـ data المطلوبة بالتالي سننفذ :

- السطر 12 : نجعل الـ node التي يشير إليها الـ prev (و هي التي تسبق الـ node التي يشير إليها الـ current) تشير إلى الـ node التي تلي الـ node التي يشير إليها الـ current. (أي : نجعل الـ node السابقة تشير إلى الـ node التالية)



- السطر 13 : نهي التابع ← الـ current و الـ prev سيتم حذفهم ← عندها ستصبح الـ node التي يشير إليها الـ current بلا pointer يشير إليها ← سيقوم الـ garbage collector بحذفها تلقائياً.



b. السطر 15 : نجعل الـ current يشير إلى الـ node التالية.

c. السطر 16 : نجعل الـ prev يشير إلى الـ node التي كان يشير إليها الـ current

(نجعل كل من المؤشرين يتقدم إلى الأمام خطوة)

- يتنفذ السطرين 15 و 16 في حال لم يتحقق الشرط في السطر 11 ، و إذا لم تكن الـ data المطلوبة موجودة في الـ Linked List ← ستنتهي الحلقة عند وصول الـ current إلى آخر node و سينتهي التابع دون حذف شيء.

5-تابع لحذف كل node تحوي ال data المطلوبة :

```

1) public void deleteEachNode(int data) {
2)   if (head == null) {
3)     return ;
4)   }
5)   while (head != null && head.data == data) {
6)     head = head.next ;
7)   }
8)   Node prev = head;
9)   Node current = head != null ? head.next : null ;
10)  while (current != null) {
11)    if (current.data == data) {
12)      prev.next = current.next ;
13)      current=current.next;
14)      continue;
15)    }
16)    current = current.next ;
17)    prev = prev.next ;
18)  }
19) }

```



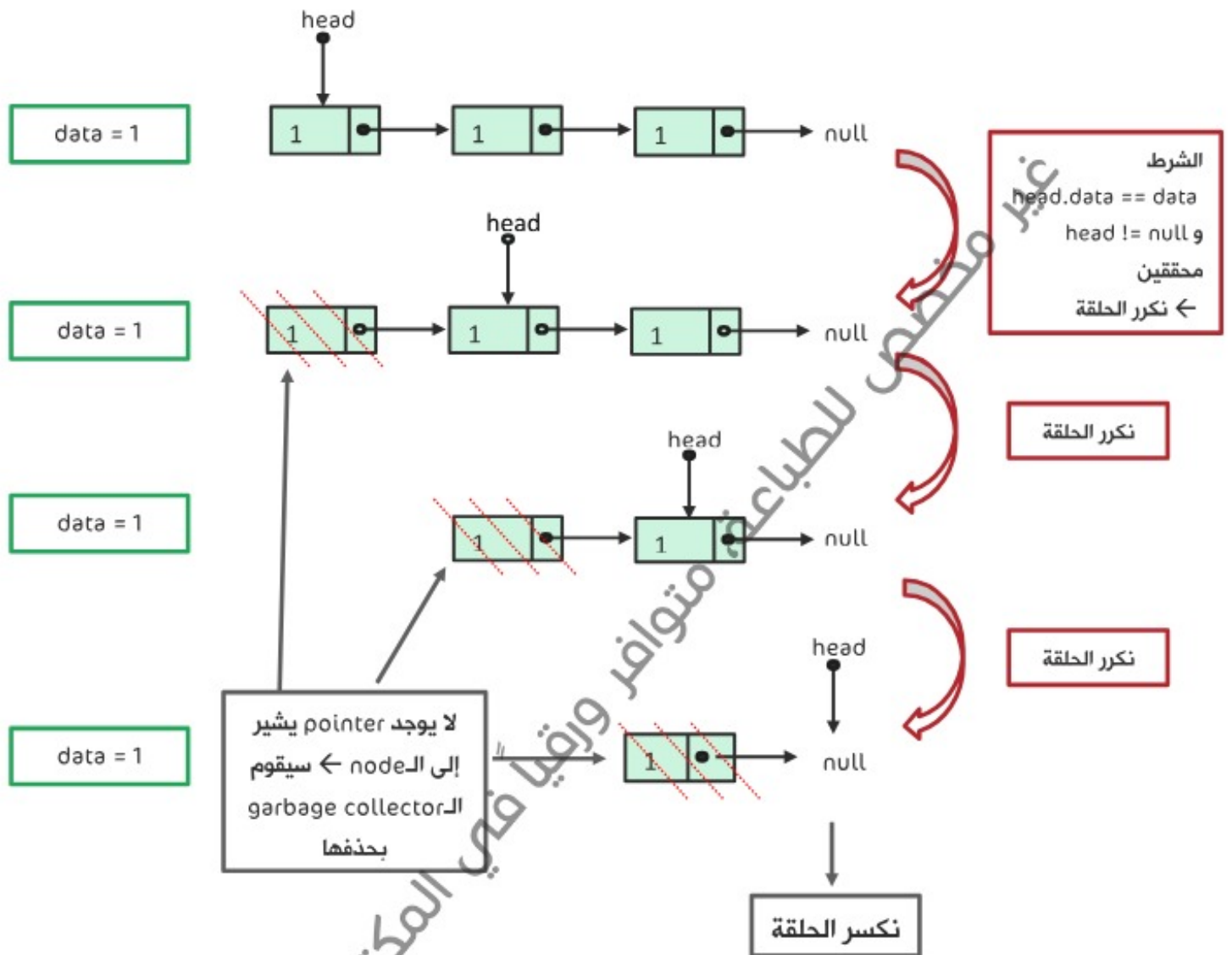
فقط سنعدل على التابع السابق قليلاً :

(1) الحالة الأولى (إذا كان ال head = null) : لم نغير شيء.

(2) الحالة الثانية (إذا كان العنصر المراد حذفه هو أول عنصر في ال Linked List) :

حولنا ال if إلى حلقة تبقى مستمرة طالما أن العنصر المطلوب يقع في البداية، و أضفنا الشرط head != null الذي يضمن لنا أن ال Linked List لم تصبح فارغة أثناء تكرار الحلقة،
تابع المثال التالي:

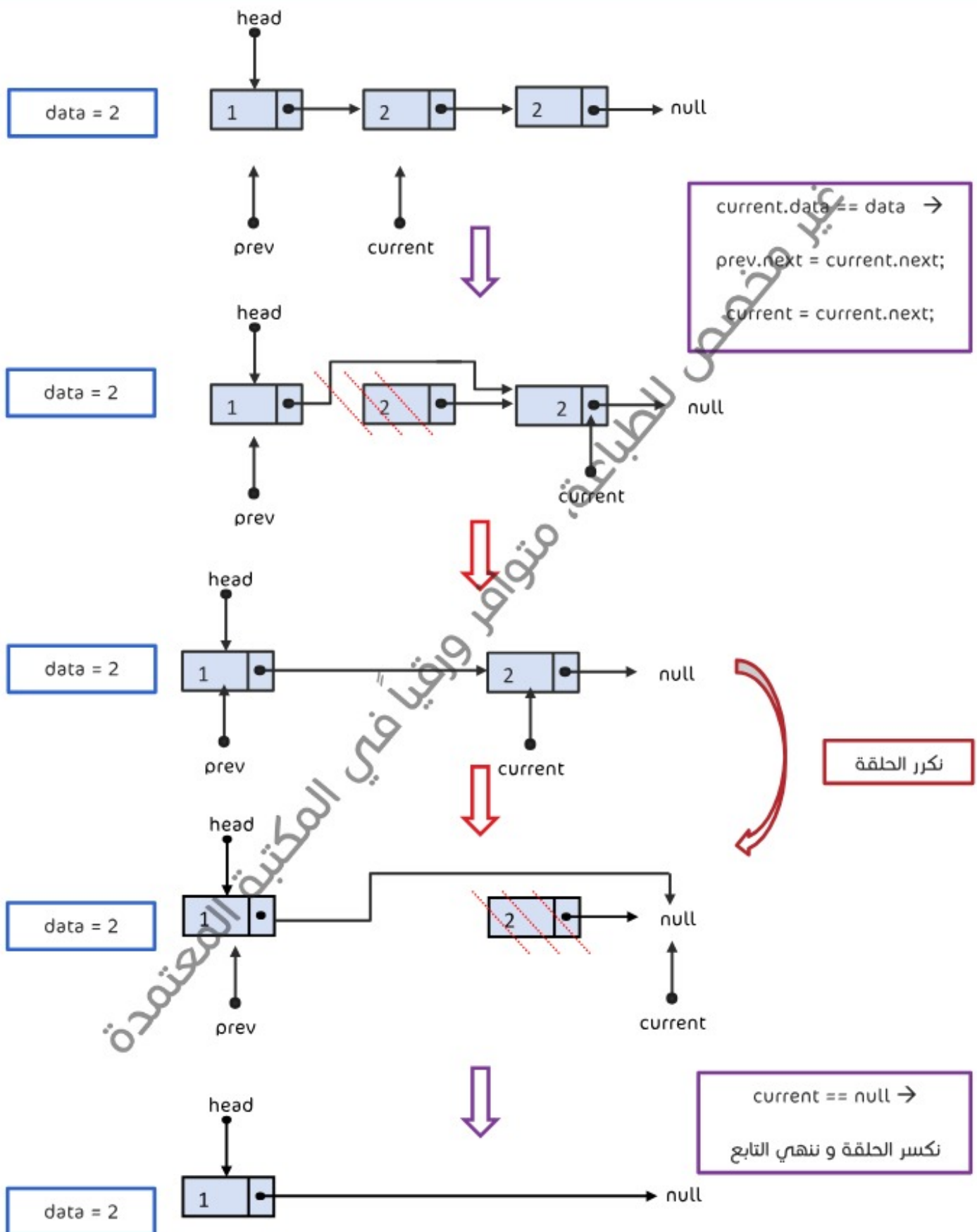
مثال :



3 الحالة الثالثة (العنصر المراد حذفه هو أي node عدا الـ node الأولى) :

هنا فقط نبدل الـ return في السطر 13 بـ continue

فيصبح : عند حذف الـ node المطلوبة لا ننهي التابع بل نعيد الحلقة حتى نحذف جميع الـ nodes التي تساوي الـ data المطلوبة في الـ Linked List ،
 تابع المثال التالي:



6-تابع يعكس ال Linked List :

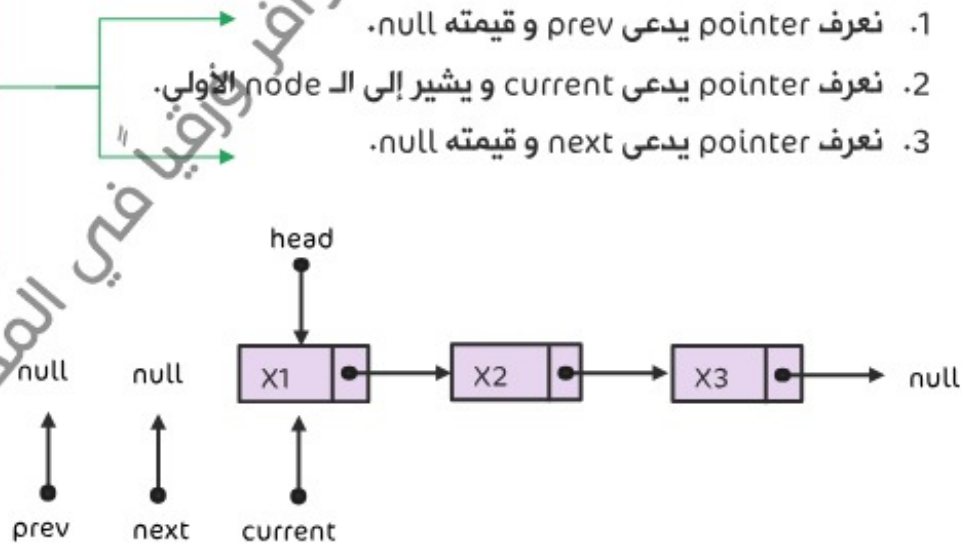
```

void reverse () {
1) Node prev = null;
2) Node current = head;
3) Node next = null;
4) while (current != null) {
5)   next = current.next;
6)   current.next = prev;
7)   prev = current;
8)   current = next;
9) }
10) head = prev;
11) }

```

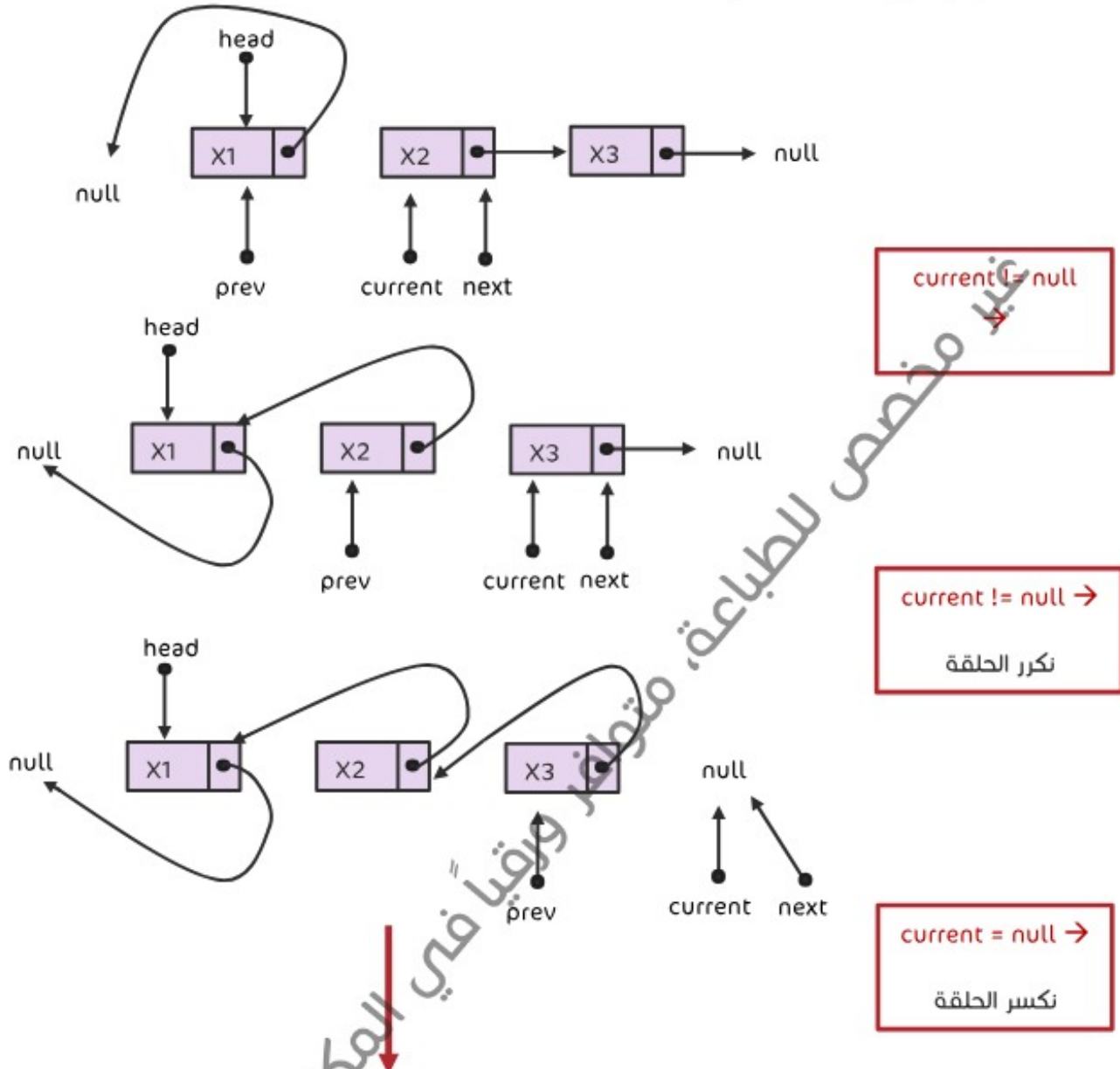
سنستخدم 3 pointers حيث سنجعل كل node تشير إلى التي قبلها
ثم نجعل ال head يشير إلى آخر node في ال linked list
و نكرر ذلك حتى نهاية ال Linked List:

ملاحظة: لم نعط ال prev
وال next قيمة في بداية الكود لأنه
من الممكن أن تكون ال Linked List
فارغة أو من عنصر واحد، لكن عند
دخول الحلقة و التحقق من الشرط
current != null
← ليست فارغة

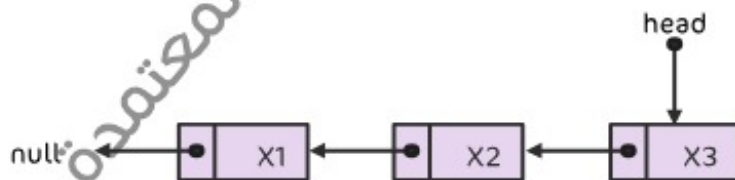


4. حلقة تمر على عناصر ال Linked List.
5. ال next يشير إلى ال node التي تلي ال node التي يشير إليها ال current.
6. ال node التي يشير إليها ال current تشير إلى ال prev.
7. ال prev يشير إلى ال node التي يشير إليها ال current.

8. الـ current يشير إلى الـ node التالية.



9. نجعل الـ head يشير إلى الـ node التي يشير إليها الـ prev.



ملاحظة :

يوجد طرق لكتابة هذا التابع باستخدام 2 pointers فقط :

أنواع الـ Linked List :

: (SLL) Singly Linked List

و هي الـ Linked List التي تعاملنا معها في محاضرتنا، لها head يشير إلى أول node فيها و آخر عنصر يشير إلى الـ null ، "الانتقال باتجاه واحد (من الـ head إلى آخر عنصر)".

حجمها في الـ RAM : يحسب بالشكل :

$$(n * 4 \text{ bytes}) + (1 \text{ bytes} * n)$$

عدد عناصر الـ Linked List

حجم الـ pointer الواحد

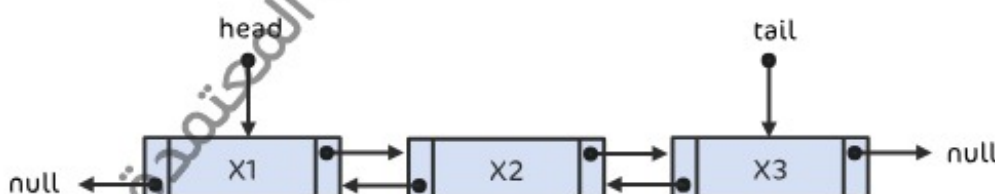
حجم العنصر الواحد

أمثلة على استخداماتها :

- To Do List (حيث أنه لدي حرية إضافة tasks إليها ← أحتاج إلى dynamic size ← اختيار الـ array هنا سيكون سيء)
- عند القيام بـ undo لشيء "Ctrl + z" (هنا نقوم بحذف آخر عنصر من الـ SLL فقط) (بشكل أدق يتم استخدام الـ stack هنا).

: (DLL) Doubly Linked List

هي Linked List لها head يشير إلى أول node فيها و tail يشير إلى آخر node فيها، و كل node فيها 2 pointers أحدهما يشير إلى الـ node التالية و الآخر يشير إلى الـ node السابقة. الانتقال يمكن أن يتم باتجاهين (من الـ head إلى الـ tail و بالعكس).



حجمها في الـ RAM : يحسب بالشكل :

$$(n * 4 \text{ bytes}) + (2 \text{ bytes} * n)$$

يوجد 2 pointers لكل عنصر

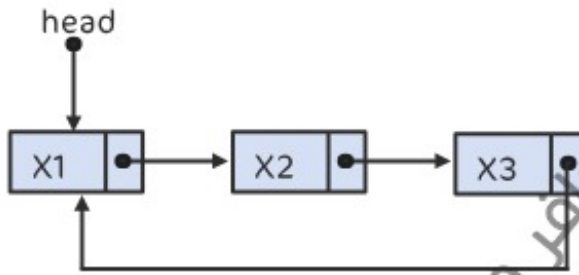
- نلاحظ أنه في كل من الـ SLL و الـ DLL تعقيد الذاكرة من المرتبة نفسها و هي $O(n)$ ، و هو تعقيد خطي ؛ لكن من المؤكد أن الـ DLL ستستهلك ذاكرة أكثر من الـ SLL بشكل عام.

أمثلة على استخداماتها :

- عندما نقوم بـ "Ctrl + z" undo و "Ctrl + y" redo (حيث يتم التنقل بين العناصر باتجاهين).
- عندما نقوم بالرجوع إلى الصفحة السابقة أو التقدم إلى الصفحة التالية في الـ browser .
- في مشغل الموسيقى عند التنقل بين الأغاني إلى الأمام و الخلف.

Circular Linked List (CLL) :

- و هي عبارة عن SLL العنصر الأخير فيها يشير إلى العنصر الأول .
- الانتقال هنا لا يزال باتجاه واحد



أمثلة على استخداماتها :

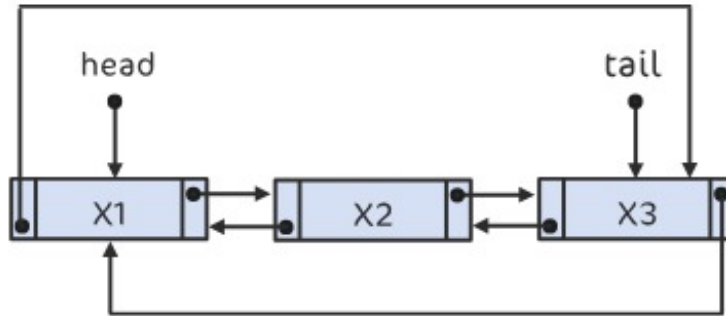
- عندما نريد مشغل الموسيقى أن يعمل بشكل حلقة حيث إذا وصل إلى آخر أغنية فسيعود إلى الأغنية الأولى.
- Traffic Light System : حيث أنه يضيء بالألوان (أخضر - أصفر - أحمر) و عندما يصل إلى العنصر الأخير يقوم بإعادة الـ loop فينتقل إلى العنصر الأول مباشرة.
- Network token passing : و هي نوع من أنواع الحلقات حيث يتم ربط الحواسيب بشكل حلقي فتنتقل المعلومات من حاسب إلى آخر بالتتالي.



يلا هانت باقي صفحة
يا وحش

(CDLL) Circular Doubly Linked List

هي DLL حيث أن العنصر الأخير فيها يشير إلى العنصر الأول و العنصر الأول يشير إلى الأخير. الانتقال يتم بالاتجاهين .



أمثلة على استخداماتها :

في مشغل الأغاني عندما يعمل بشكل حلقة و يستطيع التنقل بين الأغاني بالاتجاهين.

في كل مسألة نواجهها نقوم بتحديد العمليات التي سنقوم بها على المعطيات الخاصة بالمسألة، ثم نقارن تعقيد كل data structure بتنفيذ هذه العمليات مع ال data structures الأخرى، و بناء على ذلك نختار ال data structure المناسبة للمسألة المطروحة حيث أن كل data structure لها شروط معينة حتى يكون استخدامها أمثل.

	Array	Linked List
Insertion at Beginning	$O(n)$	$O(1)$
Insertion at End	$O(1)$	$O(n)$
Binary Search	$O(\log n)$	not available :

أمثلة :

- إذا كانت المسألة تستدعي الوصول المتكرر إلى العناصر أو القيام بقراءة العناصر بشكل كبير ← نختار ال Array لأن تعقيد الوصول فيها هو $O(1)$.
- إذا كان لدينا مسألة فيها حجم ثابت (مثل : رقعة شطرنج) ← نختار Array ، و سيكون اختيار ال Linked List سيء جداً في هذه الحالة.
- إذا كانت المسألة تستدعي إضافة عناصر بشكل متكرر ← Linked List لأن تعقيد الإضافة في البداية هو $O(1)$.

انتهت المحاضرة